

Primitive type relationships:

byte $\leq$ short $\leq$ int $\leq$ long $\leq$ float $\leq$ double
char $\leq$ int

partial order

$x \leq y \Rightarrow$ Code written for y can be used on x .

Abstraction: - reduce code complexity, *code reuse*
SEPARATION OF CONCERN $\leftrightarrow$ - compartmentalise computation
- hidden implementation
- implementer & client $\Rightarrow$ able to change implementation.

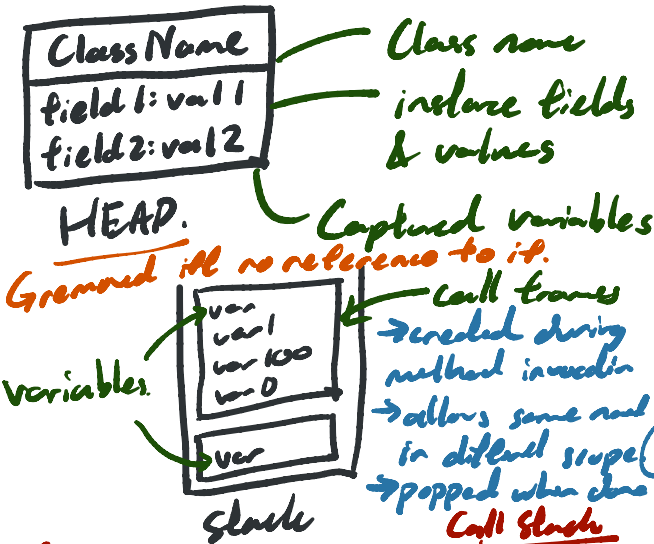
Encapsulation: - Ability to create complex data types
- Creates an abstraction barrier
- Keeps all data & functions.
- exposing ONLY certain interface.

Information Hiding: - avoid direct access to data.
further enforces the abstraction barrier
using access modifiers

Tell, don't ask: - prevent leaking implementation details to client
- tell the object what to do, rather than asking for state and doing
- Keeps encapsulation intact
- Reduced coupling.

Bridge Method: Generated when ① type extends a parameterised type ② type erasure changed signature of one or more inherited method.

Composition: - added abstraction
- models a HAS-A relation
- Be careful of aliasing.



$\emptyset$ for uninitialised vars. *Call stack*
Primitive types: call by value. *this object.*
local vars. *method args*

Inheritance: - models the IS-A relationship

Method Signature: - Method name
- No. of params.
- Type of param.
- Order of params.

Class Name: `foo(B1, B2)`
Method Descriptor: - Return Type.
`A Class Name::foo(B1, B2)`

Method Overriding: - Instance method w/ same method descriptor.

Method Overloading: - Same name, diff method signature.

Polymorphism: - Change in code behaviour w/o changing existing code. - Success code future proof.

Dynamic Binding: Class methods not supported. Step 1 taken only. CTT exec.
① **Compile Time:**
 $\rightarrow$ Determine method descriptor of invoked method w/ CTT
 $\rightarrow$ Search ALL methods in CTT that can be correctly invoked
 $\rightarrow$ Decollect w/ MOST SPECIFIC args of $M \rightarrow N$ $\checkmark$ M is more specific.
 $\rightarrow$ if no most specific: *Compilation error.*
 $\rightarrow$ Store method descriptor in bytecode.

② **Runtime:**
 $\rightarrow$ Retrieve method descriptor
 $\rightarrow$ Determine RTT of target of invocation
 $\rightarrow$ Search methods in R for matching descriptor:
 $\Rightarrow$ Starts from R, iterates to parent class.
 $\Rightarrow$ First matching method.

Liskov Substitution Principle:
Let $\phi(x)$ be a property provable about objects of type T .
Then $\phi(y)$ should be true for objects y of type S where $S \leq T$
Subclass should NOT break expectations set by superclass.
 $\Rightarrow$ B is substitutable for parent class A , pass all test cases of parent class A .
 $\Rightarrow$ Ensures any inheritance w/ method overriding does not intro bugs.

